МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка

ЗВІТ

до лабораторної роботи №10
з дисципліни «Професійний Python»

Тема: «Production Python: Docker, CI/CD та захист проєкту»

Виконав:
студент Левандівський Назар

Перевірив:
Куньо Іван Михайлович

Львів – 2026

Мета роботи

Підготувати проєкт до production-oriented використання: оформити його як installable package з `pyproject.toml`, зібрати wheel і sdist, винести конфігурацію в environment variables і прибрати secrets з коду, створити Dockerfile та .dockerignore, налаштувати CI/CD pipeline у GitHub Actions з автоматичними lint, type checking, tests і build, оформити документацію для фінального захисту.

1. Варіант і опис проєкту

Варіант 4. Система банківських рахунків. Це завершальна версія проєкту ЛР1–ЛР9: пакет `bank-account-service` / `bank_account_service`. REST API з ЛР8 збережено: клієнти, рахунки, поповнення, списання, пошук і фільтри, історія операцій, загальна сума коштів, атомарний переказ без від'ємного балансу. Persistence – SQLite, SQLAlchemy 2 і міграції Alembic з ЛР7. З ЛР9 залишено NumPy-векторизацію нарахування відсотків, яка дала найбільше прискорення: вона працює в ендпоінті прогнозу `/statistics/interest`. У ЛР10 додано конфігурацію через environment, `GET /health`, захист змін даних ключем `X-API-Key`, CLI `bank-account-service`, автоматичні міграції під час запуску, Dockerfile, CI workflow і README.

```
Lab_10/
├── pyproject.toml          метадані пакета, залежності, ruff, mypy, pytest, coverage
├── README.md, LICENSE      документація, ліцензія MIT
├── .env.example            перелік змінних середовища без секретів
├── .gitignore, .dockerignore
├── Dockerfile              двоетапна збірка, non-root, HEALTHCHECK, volume /app/data
├── alembic.ini             міграції для розробки (alembic revision/upgrade)
├── .github/workflows/ci.yml
├── src/bank_account_service/
│   ├── __init__.py         __version__ з метаданих пакета;  py.typed
│   ├── main.py             CLI bank-account-service: serve, migrate, config, --version
│   ├── config.py           Settings (pydantic-settings): environment, .env, SecretStr
│   ├── api.py              FastAPI: REST API ЛР8, /health, X-API-Key, Swagger без CDN
│   ├── database.py         engine, сесії, перевірка схеми, міграції з пакета
│   ├── models.py, repositories.py, services.py, schemas.py, value_objects.py,
│   │   exceptions.py       предметна логіка й persistence (ЛР2–ЛР7)
│   ├── interest.py         прогноз відсотків на NumPy (оптимізація з ЛР9)
│   ├── external_api.py     асинхронний клієнт bank-api-client (ЛР8)
│   ├── migrations/         Alembic: env.py, versions/0001, 0002 (ЛР7)
│   └── static/             Swagger UI (js, css)
├── tests/                  52 тести: API, клієнт, конфігурація, health, секрети, CLI
└── tools/                  check_secrets.py, ci_local.py, call_api.py
```

2. Packaging: pyproject.toml, wheel і sdist

```
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "bank-account-service"
version = "1.0.0"
description = "Bank accounts REST API: clients,
accounts, transfers, history (FastAPI + SQLite)"
readme = "README.md"
license = "MIT"
license-files = ["LICENSE"]
requires-python = ">=3.11"
authors = [{ name = "Nazar Levandivskyi" }]
keywords = ["bank", "accounts", "fastapi", "rest-api",
"sqlalchemy"]
classifiers = [
    "Development Status :: 5 - Production/Stable",
    "Framework :: FastAPI",
    "Programming Language :: Python :: 3",
    "Programming Language :: Python :: 3.11",
    "Programming Language :: Python :: 3.12",
    "Operating System :: OS Independent",
]
dependencies = [
    "fastapi>=0.115",
    "uvicorn>=0.30",
    "httpx>=0.27",
    "pydantic>=2.7",
    "pydantic-settings>=2.4",
    "sqlalchemy>=2.0",
    "alembic>=1.13",
    "numpy>=1.26",
]

[project.optional-dependencies]
dev = [
    "pytest>=8",
    "pytest-asyncio>=0.24",
    "pytest-cov>=5",
    "ruff>=0.6",
    "mypy>=1.10",
```

```
    "build>=1.2",
    "pyyaml>=6",  # tools/ci_local.py читає workflow
    "types-PyYAML",
]

[project.scripts]
bank-account-service =
"bank_account_service.main:main"
bank-api-client =
"bank_account_service.external_api:main"

[tool.setuptools.packages.find]
where = ["src"]

[tool.setuptools.package-data]
bank_account_service = ["static/*",
"migrations/script.py.mako", "py.typed"]
```

Версія 1.0.0 – за SemVer (MAJOR.MINOR.PATCH): перший стабільний реліз. `__version__` читається з метаданих встановленого пакета (`importlib.metadata`), тому версія записана лише в одному місці. Пакет має дві команди (`project.scripts`):

- `bank-account-service` – запуск, міграції, конфігурація;
- `bank-api-client` – асинхронний клієнт.

Міграції Alembic і статичні файли Swagger UI входять у wheel як package data, тому встановлений пакет працює без вихідного коду проєкту.

```
PS C:\Github\prof_python\Lab_10> python -m build
…
* Building sdist...
…
* Building wheel from sdist
…
* Building wheel...
…
Successfully built bank_account_service-1.0.0.tar.gz and bank_account_service-1.0.0-py3-none-any.whl
PS C:\Github\prof_python\Lab_10> Get-ChildItem dist
Mode   LastWriteTime       Length Name
----   -------------       ------ ----
-a---- 06.11.2026 12:29:54 463733 bank_account_service-1.0.0-py3-none-any.whl
-a---- 06.11.2026 12:29:46 464484 bank_account_service-1.0.0.tar.gz
```

Рисунок 1 – Збірка wheel і sdist (python -m build, фрагмент виводу) та вміст каталогу dist

Wheel містить 28 файлів: модулі пакета, міграції, `static/`, `py.typed` і `dist-info` (METADATA, entry_points.txt, licenses/LICENSE). Sdist (`.tar.gz`, 37 файлів) містить вихідний код, тести, README, LICENSE і `pyproject.toml`, тобто з нього можна повторити build. Метадані пакета з wheel (фрагмент METADATA):

```
Metadata-Version: 2.4
Name: bank-account-service
Version: 1.0.0
Summary: Bank accounts REST API: clients, accounts,
transfers, history (FastAPI + SQLite)
Author: Nazar Levandivskyi
License-Expression: MIT
Requires-Python: >=3.11
Requires-Dist: fastapi>=0.115
Requires-Dist: uvicorn>=0.30
Requires-Dist: httpx>=0.27
Requires-Dist: pydantic>=2.7
Requires-Dist: pydantic-settings>=2.4
Requires-Dist: sqlalchemy>=2.0
Requires-Dist: alembic>=1.13
Requires-Dist: numpy>=1.26
```

3. Конфігурація через environment і робота із secrets

Таблиця 1 – Змінні середовища застосунку

Змінна	Типово	Призначення
`APP_ENV`	development	development, test або production (Docker: production)
`DATABASE_URL`	sqlite:///bank.db	URL бази SQLAlchemy (Docker: sqlite:////app/data/bank.db у volume)
`LOG_LEVEL`	INFO	DEBUG, INFO, WARNING, ERROR
`CONFLICT_RETRIES`	3	повтори операції при конфлікті версій, 0–10
`APP_NAME`	Bank Account Service	назва в логах і Swagger

Змінна	Типово	Призначення
API_TOKEN	не задано	**secret**: якщо задано, POST, PATCH і DELETE вимагають заголовок X-API-Key

Settings (pydantic-settings) читає змінні середовища, а для локальної розробки – файл .env; змінна середовища має пріоритет над .env. Значення перевіряються під час старту: недопустимий APP_ENV чи від'ємний CONFLICT_RETRIES зупиняють сервіс з помилкою, а не приводять до некоректної роботи.

```
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8", extra="ignore")

    app_name: str = "Bank Account Service"
    app_env: Literal["development", "test", "production"] = "development"
    database_url: str = "sqlite:///bank.db"
    log_level: Literal["DEBUG", "INFO", "WARNING", "ERROR"] = "INFO"
    conflict_retries: int = Field(default=3, ge=0, le=10)
    # Якщо задано - зміни даних (POST/PATCH/DELETE) вимагають заголовок X-API-Key.
    api_token: SecretStr | None = None

    @field_validator("log_level", mode="before")
    @classmethod
    def upper_level(cls, value: object) -> object:
        return value.upper() if isinstance(value, str) else value

    @field_validator("api_token", mode="before")
    @classmethod
    def empty_token_is_unset(cls, value: object) -> object:
        # «API_TOKEN=» з .env.example означає «не задано», а не порожній пароль.
        return None if isinstance(value, str) and not value.strip() else value

    @property
    def safe_database_url(self) -> str:
        """URL БД для логів: пароль замінено на ***."""
        return make_url(self.database_url).render_as_string(hide_password=True)

@lru_cache
def get_settings() -> Settings:
    """Налаштування читаються один раз за процес (зміна середовища - перезапуск)."""
    return Settings()
```

```
# Скопіюйте у .env і заповніть: cp .env.example .env
# .env не потрапляє ні в Git (.gitignore), ні в Docker image (.dockerignore).
# Коментарі - лише окремими рядками: docker run --env-file не підтримує коментарі після значень.

APP_NAME=Bank Account Service
# development | test | production
APP_ENV=development
DATABASE_URL=sqlite:///bank.db
# DEBUG | INFO | WARNING | ERROR
LOG_LEVEL=INFO
CONFLICT_RETRIES=3
# Секрет. Якщо задано, POST/PATCH/DELETE вимагають заголовок X-API-Key з цим значенням.
# Справжнє значення - лише у .env або secrets платформи, ніколи не в Git.
API_TOKEN=
```

Secrets. У коді, README, Dockerfile і Git немає жодного справжнього пароля чи токена. Секрет API_TOKEN надходить лише із середовища: з файлу .env, з docker run --env-file або з GitHub Actions secrets. .env внесено в .gitignore і .dockerignore, тож він не потрапляє ні в репозиторій, ні в образ. Тип SecretStr не показує значення в repr і логах, пароль у DATABASE_URL маскується в лозі запуску, а команда config показує конфігурацію з прихованими секретами. Ключ X-API-Key із запиту порівнюється через secrets.compare_digest, тобто за сталий час. Сканер tools/check_secrets.py є кроком CI і шукає в проєкті присвоєння паролів і токенів, credentials в URL, приватні ключі й токени GitHub/AWS.

```
PS C:\Github\prof_python\Lab_10> python tools/check_secrets.py
Перевірено файлів: 39; підозрілих рядків: 0
```

```
user@DESKTOP-J5SKKVH:~/bank$ bank-account-service config
APP_NAME          Bank Account Service
APP_ENV           production
DATABASE_URL      sqlite:////home/user/bank/data/bank.db
LOG_LEVEL         INFO
CONFLICT_RETRIES  3
API_TOKEN         **********
```

Рисунок 2 – Перевірка проєкту на secrets і конфігурація сервісу з прихованим API_TOKEN

4. Docker: Dockerfile, .dockerignore, запуск контейнера

```
# syntax=docker/dockerfile:1

# ---------- Етап 1: збірка wheel (інструменти збірки не потрапляють у фінальний образ)
FROM python:3.12-slim AS builder
WORKDIR /build
RUN python -m pip install --no-cache-dir "build>=1.2"
COPY pyproject.toml README.md LICENSE ./
COPY src ./src
RUN python -m build --wheel --outdir /dist

# ---------- Етап 2: runtime - лише Python, встановлений wheel і його залежності
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1 \
    APP_ENV=production \
    LOG_LEVEL=INFO \
    DATABASE_URL=sqlite:////app/data/bank.db
WORKDIR /app
COPY --from=builder /dist/*.whl /tmp/
RUN python -m pip install /tmp/*.whl \
    && rm /tmp/*.whl \
    && useradd --create-home --uid 1000 appuser \
    && mkdir -p /app/data \
    && chown appuser:appuser /app/data
# Не root: процес у контейнері не має прав адміністратора.
USER appuser
# SQLite-файл - у volume, щоб дані переживали перезапуск і оновлення контейнера.
VOLUME ["/app/data"]
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s --start-period=15s --start-interval=2s --retries=3 \
    CMD python -c "import sys, urllib.request; sys.exit(urllib.request.urlopen('http://127.0.0.1:8000/health',
timeout=2).status != 200)"
# serve застосовує міграції (alembic upgrade head) і запускає uvicorn.
CMD ["bank-account-service", "serve", "--host", "0.0.0.0", "--port", "8000"]
```

Образ двоетапний (multi-stage). Етап `builder` збирає wheel, і інструменти збірки до фінального образу не потрапляють. Runtime-етап на базі `python:3.12-slim` встановлює лише wheel і його залежності. Процес працює від користувача `appuser`, не від root. SQLite-файл лежить у volume `/app/data` і переживає перезапуск та оновлення контейнера. `HEALTHCHECK` кожні 30 с перевіряє `/health`. Команда `serve` спершу застосовує міграції (автоматичні міграції при deployment), а потім запускає uvicorn у production-режимі, без `--reload`. `.dockerignore` залишає в build context лише `pyproject.toml`, README, LICENSE і `src/`, тобто без `.env`, тестів, кешів і `.git`; секретів в образі немає, вони передаються під час запуску.

```
# .dockerignore
.git
.github
.venv
.env
.env.*
__pycache__
*.pyc
.pytest_cache
.mypy_cache
.ruff_cache
.coverage
htmlcov
tests
tools
report
dist
build
*.egg-info
*.db
data
*.pdf
.ci-local
```

```
# збірка образу і перегляд
docker build -t bank-account-service:1.0.0 .
docker images bank-account-service

# запуск: порт, volume для SQLite, конфігурація через -e та --env-file (секрет лише у .env)
docker run -d --name bank -p 8000:8000 \
    -v bank_data:/app/data \
    -e APP_ENV=production -e LOG_LEVEL=INFO \
    --env-file .env \
    bank-account-service:1.0.0
docker ps                       # STATUS: Up ... (healthy) після HEALTHCHECK
curl http://localhost:8000/health
```

Обмеження. На ПК, де виконувалася робота, Docker не встановлено: у WSL Debian його немає, а для встановлення бракує прав адміністратора. Тому команди `docker build` і `docker run` не виконувалися, і знімків `docker images` та `docker ps` у звіті немає.

Runtime-етап образу перевірено без Docker, тими самими кроками, що й у Dockerfile: wheel з `dist/` встановлено в нове venv у Linux (WSL Debian), змінні задано як `ENV` і `--env-file`, сервіс запущено командою з `CMD`, стан перевірено командою з `HEALTHCHECK`. Відмінності від образу: Python 3.13 замість 3.12, каталог `~/bank/data` замість volume, звичайний користувач WSL замість `appuser`.

```
user@DESKTOP-J5SKKVH:~/bank$ python3 -m venv venv && . venv/bin/activate
user@DESKTOP-J5SKKVH:~/bank$ pip install bank_account_service-1.0.0-py3-none-any.whl | tail -n 1
Successfully installed Mako-1.4.3 MarkupSafe-3.0.4 alembic-1.20.1 annotated-doc-0.0.5 annotated-types-0.8.0 anyio-
4.16.0 bank-account-service-1.0.0 certifi-2026.10.23 click-8.5.0 fastapi-0.162.0 h11-0.16.0 httpcore-1.0.9 httpx-
0.28.1 idna-3.20 numpy-2.5.4 opentelemetry-api-1.45.2 pydantic-2.14.3 pydantic-core-2.50.4 pydantic-settings-2.16.2
python-dotenv-1.2.4 sqlalchemy-2.1.8 starlette-1.10.0 typing-extensions-4.16.0 typing-inspection-0.4.5 uvicorn-0.54.0
user@DESKTOP-J5SKKVH:~/bank$ bank-account-service --version
bank-account-service 1.0.0
user@DESKTOP-J5SKKVH:~/bank$ export APP_ENV=production LOG_LEVEL=INFO DATABASE_URL=sqlite:///$HOME/bank/data/bank.db
user@DESKTOP-J5SKKVH:~/bank$ set -a; . ./bank.env; set +a      # API_TOKEN - з файла поза Git (як --env-file)
user@DESKTOP-J5SKKVH:~/bank$ bank-account-service serve --host 0.0.0.0 --port 8000 > serve.log 2>&1 & SERVER_PID=$!
user@DESKTOP-J5SKKVH:~/bank$ cat serve.log
2026-11-06 10:29:19,359 INFO alembic.runtime.migration Context impl SQLiteImpl.
2026-11-06 10:29:19,360 INFO alembic.runtime.migration Will assume transactional DDL.
2026-11-06 10:29:19,363 INFO alembic.runtime.migration Running upgrade  -> 0001, create bank schema
2026-11-06 10:29:19,369 INFO alembic.runtime.migration Running upgrade 0001 -> 0002, add account type
2026-11-06 10:29:19,396 INFO bank_account_service migrations applied: sqlite:////home/user/bank/data/bank.db
INFO:     Started server process [165466]
INFO:     Waiting for application startup.
2026-11-06 10:29:19,759 INFO bank_account_service.api Bank Account Service 1.0.0 started: env=production,
db=sqlite:////home/user/bank/data/bank.db, api token set
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     127.0.0.1:57602 - "GET /health HTTP/1.1" 200 OK
user@DESKTOP-J5SKKVH:~/bank$ python -c "import sys, urllib.request;
sys.exit(urllib.request.urlopen('http://127.0.0.1:8000/health', timeout=2).status != 200)"; echo "exit code: $?"
exit code: 0
```

Рисунок 3 – Wheel у чистому venv (Linux), конфігурація як у Dockerfile, запуск CMD і перевірка HEALTHCHECK (exit code 0)

```
user@DESKTOP-J5SKKVH:~/bank$ curl -s localhost:8000/health
{"status":"ok","version":"1.0.0","environment":"production","database":"ok"}
user@DESKTOP-J5SKKVH:~/bank$ curl -s -w ' HTTP %{http_code}\n' -X POST localhost:8000/clients -H 'Content-Type:
application/json' -d '{"full_name": "Олена Коваленко"}'
{"detail":"invalid or missing X-API-Key"} HTTP 401
user@DESKTOP-J5SKKVH:~/bank$ api() { curl -s -w ' HTTP %{http_code}\n' -H 'Content-Type: application/json' -H "X-API-
Key: $API_TOKEN" "$@"; }
user@DESKTOP-J5SKKVH:~/bank$ api -X POST localhost:8000/clients -d '{"full_name": "Олена Коваленко"}'
{"id":1,"full_name":"Олена Коваленко"} HTTP 201
user@DESKTOP-J5SKKVH:~/bank$ api -X POST localhost:8000/accounts -d '{"client_id": 1, "number": "UA-1001",
"opening_balance": "1000.00"}'
{"id":1,"number":"UA-1001","client_id":1,"currency":"UAH","balance":"1000.00","account_type":"current"} HTTP 201
user@DESKTOP-J5SKKVH:~/bank$ api -X POST localhost:8000/accounts/1/deposit -d '{"amount": "500.00"}'
{"id":3,"account_id":1,"operation":"deposit","amount":"500.00","balance_after":"1500.00","created_at":"2026-11-
06T10:29:20.064350"} HTTP 201
user@DESKTOP-J5SKKVH:~/bank$ api -X POST localhost:8000/transfers -d '{"source_id": 1, "target_id": 2, "amount":
"300.00"}'
{"debit":
{"id":4,"account_id":1,"operation":"transfer_out","amount":"300.00","balance_after":"1200.00","created_at":"2026-11-
06T10:29:20.085824"},"credit":
{"id":5,"account_id":2,"operation":"transfer_in","amount":"300.00","balance_after":"550.00","created_at":"2026-11-
```

```
06T10:29:20.087371"}} HTTP 201
user@DESKTOP-J5SKKVH:~/bank$ api -X POST localhost:8000/transfers -d '{"source_id": 2, "target_id": 1, "amount":
"9999.00"}'
{"error":"insufficient_funds","detail":"UA-1002: недостатньо коштів (потрібно 9 999.00 UAH, є 550.00 UAH)"} HTTP 409
user@DESKTOP-J5SKKVH:~/bank$ curl -s 'localhost:8000/statistics/interest?currency=UAH&days=365'
{"currency":"UAH","days":365,"accounts":2,"total_now":"1750.00","total_after":"1933.79","interest":"183.79"}
```

Рисунок 4 – Результат /health і робота API: без X-API-Key зміни заборонені (401), з ключем – 201; переказ понад баланс – 409

Після `kill` (SIGTERM) сервіс завершився штатно: lifespan закрив пул з'єднань (у лозі «Bank Account Service stopped»). Після повторного запуску `GET /accounts` повернув ті самі рахунки з балансами 1200.00 і 550.00 UAH: дані збереглися у файлі `data/bank.db` (44 КБ), як у volume контейнера.

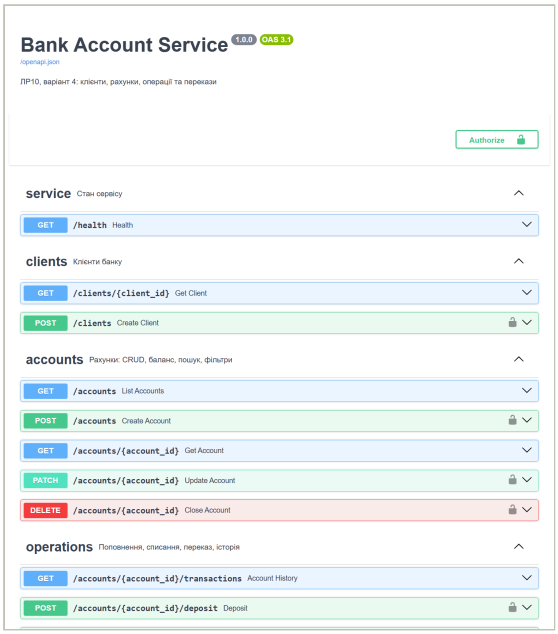


Рисунок 5 – Swagger UI (/docs) сервісу, запущеного з wheel: розділ service з /health і кнопка Authorize для X-API-Key

5. CI/CD: GitHub Actions

Workflow `.github/workflows/ci.yml` запускається на кожен push у `main`, на pull request і вручну. Права обмежено: `contents: read`. Він має два jobs:

- `quality` – матриця Python 3.11 і 3.12;
- `docker` – запускається лише після успішного `quality` (`needs`); збирає образ із тегами `ci` і коротким SHA commit, запускає контейнер і чекає відповіді `/health` (smoke test).

Секрет береться з налаштувань репозиторію (`${{ secrets.API_TOKEN }}`) і передається в контейнер як `-e API_TOKEN` без значення в команді.

```yaml
# ci.yml (фрагмент, без коментарів)
name: CI

on:
  push:
    branches: [main]
  pull_request:
  workflow_dispatch:

permissions:
  contents: read

jobs:
```

```
  quality:
    name: Lint, types, tests, build (Python ${{ matrix.python-version }})
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        python-version: ["3.11", "3.12"]
    steps:
      ...            # кроки job quality - у таблиці 2 і на рисунку 6

  docker:
    name: Docker build and smoke test
    needs: quality
    runs-on: ubuntu-latest
    env:
      API_TOKEN: ${{ secrets.API_TOKEN }}
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Build image
        run: docker build -t bank-account-service:ci -t "bank-account-service:${GITHUB_SHA::7}" .

      - name: Run container and check /health
        run: |
          docker run -d --name api -e APP_ENV=production -e API_TOKEN bank-account-service:ci
          for attempt in $(seq 1 30); do
            if docker exec api python -c "import urllib.request;
print(urllib.request.urlopen('http://127.0.0.1:8000/health').read().decode())"; then
              docker logs api
              exit 0
            fi
            sleep 1
          done
          docker logs api
          exit 1
```

GitHub Actions на GitHub не запускався, бо проєкт не публікувався в репозиторії на GitHub, тому знімка «зеленого» pipeline немає. Workflow перевірено скриптом `tools/ci_local.py`. Він читає `ci.yml` і виконує ті самі кроки `run` у чистому Linux-середовищі: свіжа копія проєкту без кешів і `.venv`, нове venv, встановлення з `pyproject.toml`. Python у WSL – 3.13, тоді як у матриці – 3.11 і 3.12; на Windows тести пройшли на Python 3.12. Job `docker` пропущено, бо Docker на ПК немає.

Таблиця 2 – Кроки workflow у локальному прогоні (Linux, Python 3.13)

Крок	**Результат**	**Час, с**
Checkout	✓ успішно	0.4
Set up Python	✓ успішно	2.8
Install	✓ успішно	20.4
Secret scan	✓ успішно	0.1
Ruff lint	✓ успішно	0.0
Ruff format check	✓ успішно	0.0
Type check (mypy --strict)	✓ успішно	11.0
Tests with coverage (fail under 85 %)	✓ успішно	8.6
Build wheel and sdist	✓ успішно	3.2
Check dist	✓ успішно	0.0
Install wheel in clean venv	✓ успішно	11.6
Upload package artifact	– пропущено (if: лише Python 3.12)	–
job docker: Checkout, Build image, Run container and check /health, Remove container	– пропущено (немає Docker)	–

```
$ python tools/check_secrets.py
Перевірено файлів: 39; підозрілих рядків: 0
$ ruff check .
All checks passed!
$ ruff format --check .
27 files already formatted
$ mypy
Success: no issues found in 26 source files
$ pytest -v --cov --cov-report=term-missing
============================= test session starts ==============================
platform linux -- Python 3.13.5, pytest-9.2.0, pluggy-1.7.0 -- /home/user/.cache/bank-ci/venv/bin/python
cachedir: .pytest_cache
…
Name                                      Stmts   Miss Branch BrPart  Cover   Missing
…
TOTAL                                       812     41    114     24    93%
6 files skipped due to complete coverage.
Required test coverage of 85.0% reached. Total coverage: 92.98%
============================== 52 passed in 6.08s ==============================
$ python -m build
…
* Building sdist...
…
* Building wheel from sdist
…
* Building wheel...
…
Successfully built bank_account_service-1.0.0.tar.gz and bank_account_service-1.0.0-py3-none-any.whl
$ ls -la dist
total 920
drwxr-xr-x  2 user user   4096 Nov  6 10:28 .
drwxr-xr-x 10 user user   4096 Nov  6 10:28 ..
-rw-r--r--  1 user user 463693 Nov  6 10:28 bank_account_service-1.0.0-py3-none-any.whl
-rw-r--r--  1 user user 463997 Nov  6 10:28 bank_account_service-1.0.0.tar.gz
$ /tmp/wheel-env/bin/bank-account-service --version
bank-account-service 1.0.0
```

Рисунок 6 – Lint, format, mypy, pytest (52 тести, покриття 92.98 % при порозі 85 %) і build у локальному прогоні job quality (фрагменти виводу)

Локальний прогін виявив реальну помилку, якої не було видно на Windows: на Python 3.13 приблизно кожен четвертий запуск тестів падав з `ResourceWarning: unclosed database`. Причина була в клієнті з ЛР8: `asyncio.gather` після відповіді 404 на один запит рахунку лишав другий запит працювати після повернення результату. Його обробку на сервері скасовували вже після закриття пулу з'єднань, а синхронна yield-залежність FastAPI при скасуванні не закриває сесію. Виправлення: у клієнті `asyncio.TaskGroup`, який скасовує сусідній запит і чекає його завершення; на сервері асинхронна залежність, що закриває сесію в `finally`; `try/finally` у lifespan. Після виправлення 25 повторних прогонів пройшли без помилок.

6. Документація, Git і production checklist

README.md має розділи Description, Features, Requirements, Installation, Configuration, Running locally, Running with Docker, API, Testing, Project structure, CI/CD, Demo scenario, Authors. Він описує предметну систему, встановлення (venv або wheel), таблицю змінних середовища, локальний запуск і запуск у Docker (з volume і `--env-file`), таблицю ендпоінтів, тести, CI/CD і сценарій демонстрації для захисту. Документацію API також генерує FastAPI: `/docs` (Swagger UI) і `/openapi.json`. Модулі й основні функції мають docstrings, а логування працює через `logging` з рівнем із `LOG_LEVEL`, без `print`.

Git. Commits у межах цієї роботи не створювалися. Зміни варто оформити такими логічними commits:

Таблиця 3 – Плановий поділ змін на commits

Commit	**Файли**
Add production configuration	config.py, main.py, .env.example, /health, X-API-Key
Add Python packaging	pyproject.toml, LICENSE, __init__.py (__version__), py.typed
Add Docker support	Dockerfile, .dockerignore
Add CI pipeline	.github/workflows/ci.yml, tools/check_secrets.py, tools/ci_local.py
Fix session leak on cancelled requests	api.py (get_service, lifespan), external_api.py (TaskGroup)
Add project documentation	README.md
Prepare release 1.0.0	версія 1.0.0, тег v1.0.0

Таблиця 4 – Production checklist

Пункт	**Стан**	**Підтвердження**
tests pass	✓	52 тести: Windows (Python 3.12) і Linux (Python 3.13), покриття 92.98 %
lint passes	✓	ruff check . і ruff format --check .
type checking passes	✓	mypy --strict (src, tests, tools)
wheel builds / sdist builds	✓	python -m build, рисунок 1
Docker image builds	✗	не перевірено: на ПК немає Docker
container starts	✗	не перевірено; runtime-етап перевірено без Docker (розд. 4)
/health returns 200	✓	у запуску з wheel та командою HEALTHCHECK
API works	✓	запити curl, 52 тести
.env is ignored / .env.example exists	✓	.gitignore, .dockerignore, .env.example
no secrets in Git	✓	tools/check_secrets.py: 0 підозрілих рядків
README updated	✓	розділи за рекомендованою структурою
CI pipeline green	частково	job quality зелений у локальному прогоні; на GitHub не запускався, job docker не виконувався

Висновки

Проєкт (варіант 4) доведено до production-oriented версії 1.0.0: пакет з `pyproject.toml`, wheel і sdist, які встановлюються в чисте venv; конфігурація через environment variables з перевіркою під час старту; секрет `API_TOKEN` поза кодом, Git і образом, сканер secrets у CI; двоетапний Dockerfile з non-root користувачем, volume, `HEALTHCHECK` і автоматичними міграціями; workflow GitHub Actions з lint, format check, mypy, тестами з порогом покриття, build і docker build зі smoke test; README зі сценарієм демонстрації. Усі 11 кроків job `quality` у локальному прогоні пройшли успішно, а сам прогін допоміг знайти й виправити витік з'єднань БД. Через відсутність Docker на ПК і публікації на GitHub не виконано збірку образу, запуск контейнера і «зелений» pipeline на GitHub; runtime-етап образу натомість перевірено запуском wheel у Linux.